

Database Systems and Laboratory

Relational Algebra

Nicola Saccomanno

`nicola.saccomanno@uniud.it`

Bachelor degrees in Internet of things, Big Data, Machine Learning, & Computer Science
DMIF, University of Udine

A.Y. 2025/2026

Introduction

- ▶ Let us now change subject, and turn to **relational query languages**
- ▶ As we shall see, they can be grouped into two large families:
 - ▶ **Procedural:** in which complex operations are specified describing all the steps that have to be performed in order to reach the final solution
 - ▶ **Declarative:** in which just the form of the desired solution is described, without specifying how such a solution should be achieved
- ▶ We will deal here with **relational algebra**, which is a **procedural language**; this will provide us with a solid basis to understand SQL, a very popular commercial declarative language for relational databases
 - ▶ As a matter of fact, the first operation performed by query optimizer in a DBMS is to translate the SQL query in equivalent relational algebra query

Relational Algebra overview

- ▶ Relational algebra is the basic set of operations for the relational model
- ▶ These operations enable a user to specify **basic retrieval requests** (or **queries**)
- ▶ The result of an operation is a *new relation*, which may have been formed from one or more *input relations*
 - ▶ This property makes the algebra “closed” (all objects in relational algebra are relations)
 - ▶ Relation algebra queries do not change the state of the database
- ▶ New relations can be further manipulated using operations of the same algebra
- ▶ A sequence of relational algebra operations forms a **relational algebra expression**
 - ▶ The result of a relational algebra expression is also a relation that represents the result of a database query (or retrieval request)

Some caveats about relational algebra

- ▶ **Relations** in the relational model are **finite** mathematical objects. Thus, **we cannot use the set-theoretic complement operation**, since it could produce infinite relations, violating closure over finite relations.
 - ▶ Substitute operation in relational algebra is **set difference**, which we will see later
 - ▶ However, set difference can behave like complement only if we have **complete information**: the absence of a tuple must mean that it is false, not unknown or missing (closed-world assumption)
- ▶ As a theoretical concept, it is important to point out that **relational algebra is non-Turing-complete**, that is, it is less powerful (i.e., we can do less) than any standard programming language (e.g., C, Python)
 - ▶ a concrete limitation is that relational algebra does not allow the expression of transitive closure (no recursion)

Relational Algebra operations overview

Relational Algebra consists of several groups of operations

- ▶ **Unary** Relational Operations

- ▶ SELECT (symbol: σ (sigma))
- ▶ PROJECT (symbol: π (pi))
- ▶ RENAME (symbol: ρ (rho))

- ▶ Relational Algebra Operations From **Set** Theory

- ▶ UNION ($\cup$), INTERSECTION ($\cap$), DIFFERENCE (or MINUS, $-$)
- ▶ CARTESIAN PRODUCT ($\times$)

- ▶ **Binary** Relational Operations

- ▶ JOIN ($\bowtie$, note that several variations of JOIN exist)
- ▶ DIVISION

- ▶ **Additional** Relational Operations

- ▶ OUTER JOINS
- ▶ AGGREGATE FUNCTIONS (These compute summary of information: for example, SUM, COUNT, AVG, MIN, MAX)

Running example

Schema

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
-------	-------	-------	------------	-------	---------	-----	--------	-----------	-----

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
-------	----------------	---------	----------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

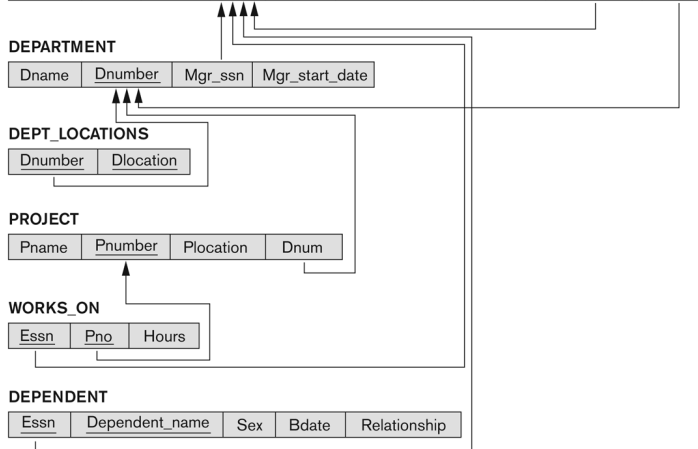
Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
-------------	------------	-------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
-------------	-----------------------	-----	-------	--------------



Running example

Instance

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1986-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Unary Relational Operations: SELECT

- ▶ The SELECT operation (denoted by σ (sigma)) is used to select a *subset* of the tuples from a relation (rows from a table) based on a **selection condition**.
 - ▶ The selection condition acts as a **filter**
 - ▶ **Keeps only those tuples that satisfy** the qualifying condition
 - ▶ Tuples satisfying the condition are *selected* whereas the other tuples are discarded (*filtered out*)
- ▶ Examples:
 - ▶ Select the EMPLOYEE tuples whose department number is 4:

$$\sigma_{\text{DNO}=4}(\text{EMPLOYEE})$$

- ▶ Select the employee tuples whose salary is greater than \$30,000:

$$\sigma_{\text{SALARY}>30,000}(\text{EMPLOYEE})$$

Unary Relational Operations: SELECT (cont.)

- ▶ In general, the *select* operation is denoted by

$$\sigma_{\langle \text{selection condition} \rangle}(R)$$

where:

- ▶ the symbol σ (sigma) is used to denote the *select* operator
- ▶ the selection condition is a **Boolean (conditional) expression** specified on the attributes of relation R , where each **term** is of the form
 - ▶ $\langle \text{attribute name} \rangle \langle \text{comparison op} \rangle \langle \text{constant value} \rangle$, or
 - ▶ $\langle \text{attribute name} \rangle \langle \text{comparison op} \rangle \langle \text{attribute name} \rangle$
 - ▶ Boolean expression example: $(\text{term}_1 \text{ AND } \text{term}_2) \text{ OR } \text{term}_3$
- ▶ tuples that make the condition **true** are selected
 - ▶ appear in the result of the operation
- ▶ tuples that make the condition **false** are filtered out
 - ▶ discarded from the result of the operation

Unary Relational Operations: SELECT (cont.)

Properties

- ▶ SELECT operation $\sigma_{\langle \text{condition} \rangle}(R)$ produces a relation S with the **same degree** as the original one, precisely with the **same schema** (same attributes) as R
- ▶ SELECT σ is **commutative**:
 - ▶ $\sigma_{\langle \text{condition1} \rangle}(\sigma_{\langle \text{condition2} \rangle}(R)) = \sigma_{\langle \text{condition2} \rangle}(\sigma_{\langle \text{condition1} \rangle}(R))$
- ▶ Because of commutativity property, a cascade (sequence) of SELECT operations may be applied in any order:
 - ▶ $\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(R))) = \sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(\sigma_{\langle \text{cond1} \rangle}(R)))$
- ▶ A cascade of SELECT operations may be replaced by a single selection with a conjunction of all the conditions:
 - ▶ $\sigma_{\langle \text{cond1} \rangle}(\sigma_{\langle \text{cond2} \rangle}(\sigma_{\langle \text{cond3} \rangle}(R))) = \sigma_{\langle \text{cond1} \rangle \text{ AND } \langle \text{cond2} \rangle \text{ AND } \langle \text{cond3} \rangle}(R)$
- ▶ The number of tuples in the result of a SELECT (**cardinality**) is always **less than (or equal to)** the number of tuples in the **input** relation R

Unary Relational Operations: PROJECT

- ▶ PROJECT operation is denoted by π (pi)
- ▶ This operation **keeps** certain *columns* (attributes) from a relation and discards the other columns.
 - ▶ PROJECT creates a **vertical partitioning**
 - ▶ The list of specified columns (attributes) is kept in each tuple
 - ▶ The other attributes in each tuple are discarded
- ▶ Example: To list each employee's first and last name and salary, the following is used:

$\pi_{\text{LNAME, FNAME, SALARY}}(\text{EMPLOYEE})$

Unary Relational Operations: PROJECT (cont.)

- ▶ The general form of the *project* operation is:

$$\pi_{\langle \text{attribute list} \rangle}(R)$$

where:

- ▶ π (pi) is the symbol used to represent the *project* operation
- ▶ $\langle \text{attribute list} \rangle$ is the desired list of attributes from relation R
- ▶ The **order** of $\langle \text{attribute list} \rangle$ is the one the attributes will have in the resulting relation
- ▶ The project operation **removes any duplicate tuples**
 - ▶ This is because the result of the *project* operation **must be a set** of tuples
 - ▶ Mathematical sets do not allow duplicate elements

Unary Relational Operations: PROJECT (contd.)

Properties

- ▶ The **degree** of the resulting operation is equal to the **number of attributes in $\langle \text{attribute list} \rangle$**
- ▶ The **cardinality** in the result of projection $\pi_{\langle \text{list} \rangle}(R)$ is **always less than or equal** to the number of tuples in R
 - ▶ If the list of attributes includes a key of R (i.e., it is a superkey), then the number of tuples in the result of PROJECT is equal to the number of tuples in R
- ▶ PROJECT is **not commutative**
- ▶ $\pi_{\langle \text{list1} \rangle}(\pi_{\langle \text{list2} \rangle}(R)) = \pi_{\langle \text{list1} \rangle}(R)$ as long as $\langle \text{list1} \rangle \subseteq \langle \text{list2} \rangle$

Examples

NL Select the tuples from all the employees who work either in department 4 and make over 25k \$ per year, or work in department 5 and make over 30k \$ per year.

Examples

NL Select the tuples from all the employees who work either in department 4 and make over 25k \$ per year, or work in department 5 and make over 30k \$ per year.

RA $\sigma_{(DNO=4 \wedge SALARY>25000) \vee (DNO=5 \wedge SALARY>30000)}(EMPLOYEE)$

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5

Examples

NL-1 Determine the first name, last name, and the salary of the employees

Examples

NL-1 Determine the first name, last name, and the salary of the employees

RA-1 $\pi_{\text{LNAME}, \text{FNAME}, \text{SALARY}}(\text{EMPLOYEE})$

NL-2 Determine the sex and the salary of the employees

Examples

NL-1 Determine the first name, last name, and the salary of the employees

RA-1 $\pi_{\text{LNAME}, \text{FNAME}, \text{SALARY}}(\text{EMPLOYEE})$

NL-2 Determine the sex and the salary of the employees

RA-2 $\pi_{\text{SEX}, \text{SALARY}}(\text{EMPLOYEE})$

Lname	Fname	Salary
Smith	John	30000
Wong	Franklin	40000
Zelaya	Alicia	25000
Wallace	Jennifer	43000
Narayan	Ramesh	38000
English	Joyce	25000
Jabbar	Ahmad	25000
Borg	James	55000

Sex	Salary
M	30000
M	40000
F	25000
F	43000
M	38000
M	25000
M	25000
M	55000

Relational Algebra Expressions

- ▶ We may want to apply several relational algebra operations one after the other
 - ▶ Either we can write the operations as a single **relational algebra expression** by nesting the operations, or
 - ▶ We can apply one operation at a time and create **intermediate result relations**.
- ▶ In the latter case, we must give names to the relations that hold the intermediate results.
 - ▶ the $\leftarrow$ symbol is the **assignment operator**

NL Retrieve the first name, last name, and salary of all employees who work in department number 5

RA Option 1: $\pi_{\text{FNAME}, \text{LNAME}, \text{SALARY}}(\sigma_{\text{DNO}=5}(\text{EMPLOYEE}))$

RA Option 2:

DEP5_EMPS $\leftarrow \sigma_{\text{DNO}=5}(\text{EMPLOYEE})$
RESULT $\leftarrow \pi_{\text{FNAME}, \text{LNAME}, \text{SALARY}}(\text{DEP5_EMPS})$

Unary Relational Operations: RENAME

- ▶ The RENAME operator is denoted by ρ (rho)
- ▶ In some cases, we may want to *rename* the attributes of a relation or the relation name or both
 - ▶ Useful when a query requires multiple operations
 - ▶ Necessary in some cases (see JOIN operation later)
- ▶ Change **both** the *relation name* to S , and the *column (attribute) names* to $B_1, B_1, \dots, B_n$

$$\rho_{S(B_1, B_2, \dots, B_n)}(R)$$

- ▶ Change the **relation name** only to S

$$\rho_S(R)$$

- ▶ Change the **column (attribute) names** to $B_1, B_1, \dots, B_n$

$$\rho_{(B_1, B_2, \dots, B_n)}(R)$$

Unary Relational Operations: RENAME (continued)

For convenience, we also use a *shorthand* for renaming attributes in an intermediate relation:

- ▶ If we write:
 - ▶ $\text{RESULT} \leftarrow \pi_{\text{FNAME}, \text{LNAME}, \text{SALARY}}(\text{DEP5_EMPS})$
 - ▶ RESULT will have the *same attribute names* as DEP5_EMPS (same attributes as EMPLOYEE)
- ▶ If we write:
 - ▶ $\text{RESULT}(\text{F}, \text{M}, \text{L}, \text{S}, \text{B}, \text{A}, \text{SX}, \text{SAL}, \text{SU}, \text{DNO}) \leftarrow (\text{DEP5_EMPS})$
 - ▶ The 10 attributes of DEP5_EMPS are *renamed* to F, M, L, S, B, A, SX, SAL, SU, DNO, respectively

Example of applying multiple operations and RENAME

$\pi_{\text{FNAME}, \text{LNAME}, \text{SALARY}}(\sigma_{\text{DNO}=5}(\text{EMPLOYEE}))$

Fname	Lname	Salary
John	Smith	30000
Franklin	Wong	40000
Ramesh	Narayan	38000
Joyce	English	25000

$\text{TEMP} \leftarrow \sigma_{\text{DNO}=5}(\text{EMPLOYEE})$

TEMP

Fname	Minit	Lname	Ssn	Bdate	Sex	Salary	Dno
John	B	Smith	123456789	1965-01-09	M	30000	5
Franklin	T	Wong	333445555	1955-12-08	M	40000	5
Ramesh	K	Narayan	666884444	1962-09-15	M	38000	5
Joyce	A	English	453453453	1972-07-31	F	25000	5

$R(\text{First_name}, \text{Last_name}, \text{Salary}) \leftarrow \pi_{\text{FNAME}, \text{LNAME}, \text{SALARY}}(\text{DEP5_EMPS})$

R

First_name	Last_name	Salary
John	Smith	30000
Franklin	Wong	40000
Ramesh	Narayan	38000
Joyce	English	25000

Relational Algebra Operations from Set Theory: UNION

UNION Operation

- ▶ Binary operation, denoted by $\cup$
- ▶ The result of $R \cup S$ is a relation that includes all tuples that are either in R or in S or in both R and S
 - ▶ $\text{degree}(R) = \text{degree}(S) = \text{degree}(R \cup S)$
- ▶ Duplicate tuples are eliminated
 - ▶ $|R \cup S| \leq |R| + |S|$ (also, $|R \cup S| \geq |R|$ and $|R \cup S| \geq |S|$)
- ▶ The two operand relations R and S must be **type compatible** (or **UNION compatible**)
 - ▶ R and S must have **same degree**, i.e., number of attributes
 - ▶ Each pair of corresponding attributes must be type compatible (have **same or compatible domains**)

Relational Algebra Operations from Set Theory

Type Compatibility

- ▶ Type Compatibility of operands is **required for the binary set operation(s)** UNION $\cup$ (also for INTERSECTION $\cap$, and SET DIFFERENCE $-$, see next slides)
- ▶ $R1(A1, A2, \dots, An)$ and $R2(B1, B2, \dots, Bn)$ are type compatible if:
 - ▶ they have the same number of attributes, and
 - ▶ the domains of corresponding attributes are type compatible (i.e. $\text{dom}(A_i) = \text{dom}(B_i)$ for $i = 1, 2, \dots, n$)
 - ▶ due to technical reason (we should introduce actual domains in the relational model), while doing exercises, we will use the **simplify assumption** that two relations are type compatible if and only if:
 1. they have the same number of attributes
 2. corresponding attributes (i.e., A_i and B_i) must have the same meaning and the same name
- ▶ thus, **renaming before the set-based operations** could be required

Relational Algebra Operations from Set Theory: UNION

Example

- NL** Retrieve the social security numbers of all employees who either work in department 5 (RESULT1 below) or *directly supervise an employee who works in department 5* (RESULT2 below)
- RA** We can use the UNION operation as follows:

Relational Algebra Operations from Set Theory: UNION

Example

- NL** Retrieve the social security numbers of all employees who either work in department 5 (RESULT1 below) or *directly supervise an employee who works in department 5* (RESULT2 below)
- RA** We can use the UNION operation as follows:

$$\text{DEP5_EMPS} \leftarrow \sigma_{\text{DNO}=5}(\text{EMPLOYEE})$$

Relational Algebra Operations from Set Theory: UNION

Example

NL Retrieve the social security numbers of all employees who either work in department 5 (RESULT1 below) or *directly supervise an employee who works in department 5* (RESULT2 below)

RA We can use the UNION operation as follows:

$$\begin{aligned}\text{DEP5_EMPS} &\leftarrow \sigma_{\text{DNO}=5}(\text{EMPLOYEE}) \\ \text{RESULT1} &\leftarrow \pi_{\text{SSN}}(\text{DEP5_EMPS})\end{aligned}$$

Relational Algebra Operations from Set Theory: UNION

Example

NL Retrieve the social security numbers of all employees who either work in department 5 (RESULT1 below) or *directly supervise an employee who works in department 5* (RESULT2 below)

RA We can use the UNION operation as follows:

$$\text{DEP5_EMPS} \leftarrow \sigma_{\text{DNO}=5}(\text{EMPLOYEE})$$
$$\text{RESULT1} \leftarrow \pi_{\text{SSN}}(\text{DEP5_EMPS})$$
$$\text{RESULT2}(\text{SSN}) \leftarrow \pi_{\text{SUPERSSN}}(\text{DEP5_EMPS})$$

Relational Algebra Operations from Set Theory: UNION

Example

- NL** Retrieve the social security numbers of all employees who either work in department 5 (RESULT1 below) or *directly supervise an employee who works in department 5* (RESULT2 below)
- RA** We can use the UNION operation as follows:

$$\begin{aligned}\text{DEP5_EMPS} &\leftarrow \sigma_{\text{DNO}=5}(\text{EMPLOYEE}) \\ \text{RESULT1} &\leftarrow \pi_{\text{SSN}}(\text{DEP5_EMPS}) \\ \text{RESULT2}(\text{SSN}) &\leftarrow \pi_{\text{SUPERSSN}}(\text{DEP5_EMPS}) \\ \text{RESULT} &\leftarrow \text{RESULT1} \cup \text{RESULT2}\end{aligned}$$

- The union operation produces the tuples that are in either RESULT1 or RESULT2 or both

RESULT1	RESULT2	RESULT														
<table><tr><th>Ssn</th></tr><tr><td>123456789</td></tr><tr><td>333445555</td></tr><tr><td>666884444</td></tr><tr><td>453453453</td></tr></table>	Ssn	123456789	333445555	666884444	453453453	<table><tr><th>Ssn</th></tr><tr><td>333445555</td></tr><tr><td>888665555</td></tr></table>	Ssn	333445555	888665555	<table><tr><th>Ssn</th></tr><tr><td>123456789</td></tr><tr><td>333445555</td></tr><tr><td>666884444</td></tr><tr><td>453453453</td></tr><tr><td>888665555</td></tr></table>	Ssn	123456789	333445555	666884444	453453453	888665555
Ssn																
123456789																
333445555																
666884444																
453453453																
Ssn																
333445555																
888665555																
Ssn																
123456789																
333445555																
666884444																
453453453																
888665555																

Relational Algebra Operations from Set Theory: INTERSECTION

INTERSECTION

- ▶ Binary operation denoted by $\cap$
- ▶ The result of the operation $R \cap S$ is a relation that includes all tuples that are in both R and S
 - ▶ The attribute names in the result will be the same as the attribute names in R (mandatory, in our setting)
- ▶ The two operand relations R and S must be **type compatible**

STUDENT		INSTRUCTOR		STUDENT $\cap$ INSTRUCTOR	
F _n	L _n	Fname	Lname	F _n	L _n
Susan	Yao	John	Smith	Susan	Yao
Ramesh	Shah	Ricardo	Browne	Ramesh	Shah
Johnny	Kohler	Susan	Yao		
Barbara	Jones	Francis	Johnson		
Amy	Ford	Ramesh	Shah		
Jimmy	Wang				
Ernest	Gilbert				

- ▶ N.B.: Here we should have renamed the attributes of the INSTRUCTOR relation before performing the operations!

Relational Algebra Operations from Set Theory: SET DIFFERENCE

SET DIFFERENCE (also called MINUS or EXCEPT)

- ▶ Binary operation denoted by $-$
- ▶ The result of $R - S$ is a relation that includes all tuples that are in R but not in S
 - ▶ The attribute names in the result will be the same as the attribute names in R (mandatory, in our setting)
- ▶ The two operand relations R and S must be **type compatible**

STUDENT

F _n	L _n
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

F _{name}	L _{name}
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

STUDENT- INSTRUCTOR

F _n	L _n
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR - STUDENT

F _{name}	L _{name}
John	Smith
Ricardo	Browne
Francis	Johnson

- ▶ N.B.: Here we should have renamed the attributes of the INSTRUCTOR relation before performing the operation!

Some properties of UNION, INTERSECT, and DIFFERENCE

- ▶ Notice that both **union** and **intersection** are **commutative** operations; that is
 - ▶ $R \cup S = S \cup R$, and $R \cap S = S \cap R$
- ▶ Both **union** and **intersection** can be treated as n-ary operations applicable to any number of relations as both are **associative** operations; that is
 - ▶ $R \cup (S \cup T) = (R \cup S) \cup T$
 - ▶ $(R \cap S) \cap T = R \cap (S \cap T)$
- ▶ The **set difference** operation is **not commutative** and **not associative**; that is, in general
 - ▶ $R - S \neq S - R$
 - ▶ $(R - S) - T \neq R - (S - T)$

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT

CARTESIAN (or CROSS) PRODUCT Operation

- ▶ It is used to **combine tuples from two relations** in a combinatorial fashion
- ▶ Denoted by $R(A_1, A_2, \dots, A_n) \times S(B_1, B_2, \dots, B_m)$
- ▶ Result is a relation Q with **degree** $n + m$ attributes:

$$Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m),$$

in that order

- ▶ The resulting relation has one tuple for each combination of tuples—one from R and one from S
- ▶ Hence, if R has n_R tuples (denoted as $|R| = n_R$), and S has n_S tuples, then $R \times S$ will have $n_R \cdot n_S$ tuples. What if $n_R = 0$?
- ▶ The two operands do **NOT** have to be **type compatible**. However, the name of the attributes in R must differ from those in S
- ▶ Commutativity and associativity are granted as we have named attributes (vs position-based semantic)

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT (cont.)

- ▶ Generally, CROSS PRODUCT is not a meaningful operation
 - ▶ Can become meaningful when followed by other operations
- ▶ Example (not meaningful):

```
FEMALE_EMPS  $\leftarrow \sigma_{\text{SEX}='F'}(\text{EMPLOYEE})$   
EMP_NAMES  $\leftarrow \pi_{\text{FNAME}, \text{LNAME}, \text{SSN}}(\text{FEMALE_EMPS})$   
EMP_DEPENDENTS  $\leftarrow \text{EMP_NAMES} \times \text{DEPENDENT}$ 
```

- ▶ EMP_DEPENDENTS will contain every combination of EMP_NAMES and DEPENDENT, whether or not they are actually related

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT (cont.)

- ▶ To keep only combinations where the DEPENDENT is related to the EMPLOYEE, we add a SELECT operation as follows
- ▶ Example (meaningful): Find the name of female employees and their dependents

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT (cont.)

- ▶ To keep only combinations where the DEPENDENT is related to the EMPLOYEE, we add a SELECT operation as follows
- ▶ Example (meaningful): Find the name of female employees and their dependents

$$\text{FEMALE_EMPS} \leftarrow \sigma_{\text{SEX}='F'}(\text{EMPLOYEE})$$

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT (cont.)

- ▶ To keep only combinations where the DEPENDENT is related to the EMPLOYEE, we add a SELECT operation as follows
- ▶ Example (meaningful): Find the name of female employees and their dependents

$$\text{FEMALE_EMPS} \leftarrow \sigma_{\text{SEX}='F'}(\text{EMPLOYEE})$$
$$\text{EMP_NAMES} \leftarrow \pi_{\text{FNAME}, \text{LNAME}, \text{SSN}}(\text{FEMALE_EMPS})$$

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT (cont.)

- ▶ To keep only combinations where the DEPENDENT is related to the EMPLOYEE, we add a SELECT operation as follows
- ▶ Example (meaningful): Find the name of female employees and their dependents

```
FEMALE_EMPS  $\leftarrow$   $\sigma_{\text{SEX}='F'}$ (EMPLOYEE)  
EMP_NAMES  $\leftarrow$   $\pi_{\text{FNAME}, \text{LNAME}, \text{SSN}}$ (FEMALE_EMPS)  
EMP_DEPENDENTS  $\leftarrow$  EMP_NAMES  $\times$  DEPENDENT
```

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT (cont.)

- ▶ To keep only combinations where the DEPENDENT is related to the EMPLOYEE, we add a SELECT operation as follows
- ▶ Example (meaningful): Find the name of female employees and their dependents

```
FEMALE_EMPS  $\leftarrow \sigma_{SEX='F'}(EMPLOYEE)$   
EMP_NAMES  $\leftarrow \pi_{FNAME, LNAME, SSN}(FEMALE_EMPS)$   
EMP_DEPENDENTS  $\leftarrow EMP\_NAMES \times DEPENDENT$   
ACTUAL_DEPS  $\leftarrow \sigma_{SSN=ESSN}(EMP\_DEPENDENTS)$ 
```

Relational Algebra Operations from Set Theory: CARTESIAN PRODUCT (cont.)

- ▶ To keep only combinations where the DEPENDENT is related to the EMPLOYEE, we add a SELECT operation as follows
- ▶ Example (meaningful): Find the name of female employees and their dependents

```
FEMALE_EMPS  $\leftarrow \sigma_{SEX='F'}(EMPLOYEE)$   
EMP_NAMES  $\leftarrow \pi_{FNAME, LNAME, SSN}(FEMALE_EMPS)$   
EMP_DEPENDENTS  $\leftarrow EMP\_NAMES \times DEPENDENT$   
ACTUAL_DEPS  $\leftarrow \sigma_{SSN=ESSN}(EMP\_DEPENDENTS)$   
RESULT  $\leftarrow \pi_{FNAME, LNAME, DEPENDENT\_NAME}(ACTUAL\_DEPS)$ 
```

- ▶ RESULT will now contain the name of female employees and their dependents
- ▶ Note that the example is meaningful because the SELECT operation was applied between two attributes that are actually related: they are linked by means of a foreign key

The CARTESIAN PRODUCT (CROSS PRODUCT) operation

Graphical example

$$\text{FEMALE_EMPS} \leftarrow \sigma_{\text{SEX}='F'}(\text{EMPLOYEE})$$
$$\text{EMP_NAMES} \leftarrow \pi_{\text{FNAME}, \text{LNAME}, \text{SSN}}(\text{FEMALE_EMPS})$$

FEMALE_EMPS

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn
Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555

EMP_NAMES

Fname	Lname	Ssn
Alicia	Zelaya	999887777
Jennifer	Wallace	987654321
Joyce	English	453453453

The CARTESIAN PRODUCT (CROSS PRODUCT) operation

Graphical example (cont.)

EMP_DEPENDENTS $\leftarrow$ EMPNAMES $\times$ DEPENDENT

EMP_DEPENDENTS							
Fname	Lname	Ssn	Essn	Dependent_name	Sex	Bdate	...
Alicia	Zelaya	999887777	333445555	Alice	F	1986-04-05	...
Alicia	Zelaya	999887777	333445555	Theodore	M	1983-10-25	...
Alicia	Zelaya	999887777	333445555	Joy	F	1958-05-03	...
Alicia	Zelaya	999887777	987654321	Abner	M	1942-02-28	...
Alicia	Zelaya	999887777	123456789	Michael	M	1988-01-04	...
Alicia	Zelaya	999887777	123456789	Alice	F	1988-12-30	...
Alicia	Zelaya	999887777	123456789	Elizabeth	F	1967-05-05	...
Jennifer	Wallace	987654321	333445555	Alice	F	1986-04-05	...
Jennifer	Wallace	987654321	333445555	Theodore	M	1983-10-25	...
Jennifer	Wallace	987654321	333445555	Joy	F	1958-05-03	...
Jennifer	Wallace	987654321	987654321	Abner	M	1942-02-28	...
Jennifer	Wallace	987654321	123456789	Michael	M	1988-01-04	...
Jennifer	Wallace	987654321	123456789	Alice	F	1988-12-30	...
Jennifer	Wallace	987654321	123456789	Elizabeth	F	1967-05-05	...
Joyce	English	453453453	333445555	Alice	F	1986-04-05	...
Joyce	English	453453453	333445555	Theodore	M	1983-10-25	...
Joyce	English	453453453	333445555	Joy	F	1958-05-03	...
Joyce	English	453453453	987654321	Abner	M	1942-02-28	...
Joyce	English	453453453	123456789	Michael	M	1988-01-04	...
Joyce	English	453453453	123456789	Alice	F	1988-12-30	...
Joyce	English	453453453	123456789	Elizabeth	F	1967-05-05	...

The CARTESIAN PRODUCT (CROSS PRODUCT) operation

Graphical example (cont.)

$\text{ACTUAL_DEPS} \leftarrow \sigma_{\text{SSN}=\text{ESSN}}(\text{EMP_DEPENDENTS})$
 $\text{RESULT} \leftarrow \pi_{\text{FNAME}, \text{LNAME}, \text{DEPENDENT_NAME}}(\text{ACTUAL_DEPS})$

ACTUAL_DEPENDENTS

Fname	Lname	Ssn	Essn	Dependent_name	Sex	Bdate	...
Jennifer	Wallace	987654321	987654321	Abner	M	1942-02-28	...

RESULT

Fname	Lname	Dependent_name
Jennifer	Wallace	Abner

Compare tuples from different tables

- ▶ SELECT is an operator with arity 1
- ▶ To compare tuples from different tables:
 1. We need to merge the two tables, via the CARTESIAN PRODUCT
 2. We can apply SELECT on the relation resulting from the merge, specifying a selection condition that involves attributes from both the original relations

How to compare tuples from the same table?

- ▶ SELECT is an operator with arity 1
- ▶ To compare tuples from different tables:
 1. We need to merge the two copies of the same table, via RENAME and CARTESIAN PRODUCT
 2. We can apply SELECT on the relation resulting from the merge

NL Find those employees that belong to the same department

How to compare tuples from the same table?

- ▶ SELECT is an operator with arity 1
- ▶ To compare tuples from different tables:
 1. We need to merge the two copies of the same table, via RENAME and CARTESIAN PRODUCT
 2. We can apply SELECT on the relation resulting from the merge

NL Find those employees that belong to the same department

→ Find the pairs $(\text{employee}_1, \text{employee}_2)$ s.t. the employee_1 belongs to the same department as employee_2

How to compare tuples from the same table?

- ▶ SELECT is an operator with arity 1
- ▶ To compare tuples from different tables:
 1. We need to merge the two copies of the same table, via RENAME and CARTESIAN PRODUCT
 2. We can apply SELECT on the relation resulting from the merge

NL Find those employees that belong to the same department

- Find the pairs $(\text{employee}_1, \text{employee}_2)$ s.t. the employee_1 belongs to the same department as employee_2 (and remove duplicates)

How to compare tuples from the same table?

- ▶ SELECT is an operator with arity 1
- ▶ To compare tuples from different tables:
 1. We need to merge the two copies of the same table, via RENAME and CARTESIAN PRODUCT
 2. We can apply SELECT on the relation resulting from the merge

NL Find those employees that belong to the same department

→ Find the pairs $(\text{employee}_1, \text{employee}_2)$ s.t. the employee_1 belongs to the same department as employee_2 (and remove duplicates)

RA

```
EMPLOYEE_1(SSN_1, FNAME_1, ..., DNO_1) ← EMPLOYEE  
TEMP ← EMPLOYEE × EMPLOYEE_1  
RESULT ←  $\sigma_{(DNO = DNO_1) \text{ AND } (SSN < SSN_1)}$ (TEMP)
```

Exercise

NL Return the employees that have two or more dependents

Exercise

NL Return the employees that have two or more dependents

→ Return the employees that have at least two dependents

Exercise

- NL Return the employees that have two or more dependents
 - Return the employees that have at least two dependents
 - ▶ We need to count in a bound manner!

Exercise

NL Return the employees that have two or more dependents

→ Return the employees that have at least two dependents

► We need to count in a bound manner!

RA

$$\begin{aligned} \text{DEPENDENT_1}(\text{ESSN_1}, \text{DEPENDENT_NAME_1}, \dots) &\leftarrow \text{DEPENDENT} \\ \text{AT_LEAST_2} &\leftarrow \sigma_{\substack{\text{ESSN} = \text{ESSN_1} \\ \text{AND} \\ \text{DEPEN_NAME} \neq \text{DEPEN_NAME_1}}} (\text{DEPENDENT} \times \text{DEPENDENT_1}) \end{aligned}$$

NL Return the employees that have at least three dependents

RA

$$\begin{aligned} \text{DEPENDENT_2}(\text{ESSN_2}, \text{DEPENDENT_NAME_2}, \dots) &\leftarrow \text{DEPENDENT} \\ \text{AT_LEAST_3} &\leftarrow \sigma_{\substack{\text{ESSN} = \text{ESSN_2} \\ \text{AND} \\ \text{DEPEN_NAME} \neq \text{DEPEN_NAME_2} \\ \text{AND} \\ \text{DEPEN_NAME_1} \neq \text{DEPEN_NAME_2}}} (\text{AT_LEAST_2} \times \text{DEPENDENT_2}) \end{aligned}$$

Binary Relational Operations: JOIN

JOIN Operation (denoted by $\bowtie$)

- ▶ The sequence of CARTESIAN PRODUCT followed by SELECT is used quite commonly to **identify and select related tuples from two relations**
- ▶ A special operation, called JOIN, combines this sequence into a single operation
- ▶ This operation is very important for any relational database with more than a single relation, because it allows us to *combine related tuples* from various relations
- ▶ The general form of a join operation on two relations $R(A_1, A_2, \dots, A_n)$ and $S(B_1, B_2, \dots, B_m)$ is:

$$R \bowtie_{\langle \text{join condition} \rangle} S$$

$$(\text{equivalent to } \sigma_{\langle \text{join condition} \rangle}(R \times S))$$

where R and S can be any relations that result from general *relational algebra expressions*.

Binary Relational Operations: JOIN (cont.)

- ▶ Example: Suppose that we want to retrieve the name of the manager of each department.
 - ▶ To get the manager's name, we need to combine each DEPARTMENT tuple with the EMPLOYEE tuple whose SSN value matches the MGRSSN value in the department tuple
 - ▶ We do this by using the join $\bowtie$ operation.

$\text{DEPT_MGR} \leftarrow \text{DEPARTMENT} \bowtie_{\text{MGRSSN}=\text{SSN}} \text{EMPLOYEE}$

- ▶ $\text{MGRSSN} = \text{SSN}$ is the join condition
 - ▶ Combines each department record with the employee who manages the department
 - ▶ The join condition can also be specified as $\text{DEPARTMENT.MGRSSN} = \text{EMPLOYEE.SSN}$

DEPT_MGR								
Dname	Dnumber	Mgr_ssn	...	Fname	Minit	Lname	Ssn	...
Research	5	333445555	...	Franklin	T	Wong	333445555	...
Administration	4	987654321	...	Jennifer	S	Wallace	987654321	...
Headquarters	1	888665555	...	James	E	Borg	888665555	...

Binary Relational Operations: JOIN (cont.)

Properties

- ▶ Consider the following JOIN operation:

$$R(A_1, A_2, \dots, A_n) \bowtie_{R.A_i=S.B_j} S(B_1, B_2, \dots, B_m)$$

- ▶ Result is a relation Q with **degree $n + m$** attributes:
 - ▶ $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order
- ▶ The resulting relation state has one tuple for each combination of tuples— r from R and s from S , but *only if they satisfy the join condition* $r[A_i] = s[B_j]$
- ▶ Hence, if R has n_R tuples, and S has n_S tuples, then the join result will generally have *less than* $n_R \times n_S$ tuples
- ▶ Only related tuples (based on the join condition) will appear in the result

Different types of JOIN

Theta-join

- ▶ The general case of JOIN operation is called a Theta-join:

$$R \bowtie_{\Theta} S$$

- ▶ The join condition is called *theta*
- ▶ *Theta* can be **any general boolean expression on the attributes of R and S** ; for example:
 - ▶ $R.A_i < S.B_j$ AND $(R.A_k = S.B_l$ OR $R.A_p < S.B_q)$
- ▶ Most join conditions involve one or more equality conditions “AND”ed together; for example:
 - ▶ $R.A_i = S.B_j$ AND $R.A_k = S.B_l$ AND $R.A_p = S.B_q$

Different types of JOIN

EQUIJOIN

- ▶ The most common use of join involves join conditions with *equality comparisons* and *boolean conjunctions only*
- ▶ Such a join, where the only comparison operator used is $=$, is called EQUIJOIN.
 - ▶ $R.A_i = S.B_j$ AND $R.A_k = S.B_l$ AND $R.A_p = S.B_q$
- ▶ In the result of an EQUIJOIN we always have one or more pairs of attributes (whose names need not be identical) that have identical values in every tuple.
- ▶ The JOIN seen in the previous example were EQUIJOIN

Different types of JOIN

NATURAL JOIN

- ▶ Another variation of JOIN called NATURAL JOIN — denoted by $\bowtie$ without any condition — was created to **get rid of the second (superfluous) attribute resulting from an EQUIJOIN** condition.
 - ▶ i.e., one of each pair of attributes with identical values is superfluous
- ▶ The standard definition of natural join requires that the two join attributes, or each pair of corresponding join attributes, **have the same name** in both relations
- ▶ If this is not the case, a renaming operation should be applied first
- ▶ Assuming $R(\mathcal{X})$ and $S(\mathcal{Y})$, with $\mathcal{X}$ and $\mathcal{Y}$ sets of attributes
 - ▶ $\text{degree}(R \bowtie S) = \text{degree}(R) + \text{degree}(S) - |\mathcal{X} \cap \mathcal{Y}|$
 - ▶ What if $|\mathcal{X} \cap \mathcal{Y}| = 0$? Then,

Different types of JOIN

NATURAL JOIN

- ▶ Another variation of JOIN called NATURAL JOIN — denoted by $\bowtie$ without any condition — was created to **get rid of the second (superfluous) attribute resulting from an EQUIJOIN** condition.
 - ▶ i.e., one of each pair of attributes with identical values is superfluous
- ▶ The standard definition of natural join requires that the two join attributes, or each pair of corresponding join attributes, **have the same name** in both relations
- ▶ If this is not the case, a renaming operation should be applied first
- ▶ Assuming $R(\mathcal{X})$ and $S(\mathcal{Y})$, with $\mathcal{X}$ and $\mathcal{Y}$ sets of attributes
 - ▶ $\text{degree}(R \bowtie S) = \text{degree}(R) + \text{degree}(S) - |\mathcal{X} \cap \mathcal{Y}|$
 - ▶ What if $|\mathcal{X} \cap \mathcal{Y}| = 0$? Then, $R \bowtie S = R \times S$

Different types of JOIN

NATURAL JOIN – Examples

- ▶ Find the location associated with each department. This can be done via a natural join on the DNUMBER attributes of DEPARTMENT and DEPT_LOCATIONS, for which it is sufficient to write:

$$\text{DEPT_LOCS} \leftarrow \text{DEPARTMENT} \bowtie \text{DEPT_LOCATIONS}$$

- ▶ The only attribute with the same name is DNUMBER
- ▶ An implicit join condition is created based on this attribute:

$$\text{DEPARTMENT.DNUMBER} = \text{DEPT_LOCATIONS.DNUMBER}$$

- ▶ Another example:

$$Q \leftarrow R(A, B, C, D) \bowtie S(C, D, E)$$

- ▶ The implicit join condition includes *each pair of attributes with the same name*, “AND”ed together: $R.C = S.C$ AND $R.D = S.D$
- ▶ Result keeps only one attribute of each such pair: $Q(A, B, C, D, E)$

Example of NATURAL JOIN operation

NL For every project, find the info about the associated dept.

RA

$DEPT \leftarrow \rho_{(Dname, Dnum, Mgr_ssn, Mgr_start_date)}(DEPARTMENT)$
 $PROJ_DEPT \leftarrow PROJECT \bowtie DEPT$

PROJ_DEPT

Pname	Pnumber	Plocation	Dnum	Dname	Mgr_ssn	Mgr_start_date
ProductX	1	Bellaire	5	Research	333445555	1988-05-22
ProductY	2	Sugarland	5	Research	333445555	1988-05-22
ProductZ	3	Houston	5	Research	333445555	1988-05-22
Computerization	10	Stafford	4	Administration	987654321	1995-01-01
Reorganization	20	Houston	1	Headquarters	888665555	1981-06-19
Newbenefits	30	Stafford	4	Administration	987654321	1995-01-01

$DEPT_LOCS \leftarrow DEPARTMENT \bowtie DEPT_LOCATIONS$

DEPT_LOCS

Dname	Dnumber	Mgr_ssn	Location
Headquarters	1	888665555	Houston
Administration	4	987654321	Stafford
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston

Query with universal conditions

The case of "all" / subset

NL Find the employees that work on **all** projects employee "ROSSI" works on

- ▶ Candidates: all employees
- ▶ For every project p , if p is worked by "ROSSI" then also the candidate employee should work on p

Query with universal conditions

The case of "all" / subset

NL Find the employees that work on **all** projects employee "ROSSI" works on

- ▶ Candidates: all employees
- ▶ For every project p , if p is worked by "ROSSI" then also the candidate employee should work on p
 - ▶ From set perspective: $\text{projects_by_rossi} \subseteq \text{projects_by_candidate}$

Query with universal conditions

The case of "all" / subset

NL Find the employees that work on **all** projects employee "ROSSI" works on

- ▶ Candidates: all employees
- ▶ For every project p , if p is worked by "ROSSI" then also the candidate employee should work on p
 - ▶ From set perspective: $\text{projects_by_rossi} \subseteq \text{projects_by_candidate}$
- ▶ No good: a candidate employee for whom there exists a project worked on by "ROSSI" that the candidate employee does not work on
 - ▶ From set perspective: $\text{projects_by_rossi} \not\subseteq \text{projects_by_noGoodCandidate}$

Query with universal conditions I

The case of "all" / subset, $\text{projects_by_rossi} \subseteq \text{projects_by_candidate}$ - operationally

1. Identify the set of candidates (e.g., all the employees)

$$\text{CANDIDATES}(\text{ESSN}) \leftarrow \pi_{\text{SSN}}(\text{EMPLOYEE})$$

2. Build the table that represents the set of minimal requirements (i.e., at least) each candidate must satisfy in order to be considered a good candidate (e.g., the set of projects worked by "ROSSI")

$$\begin{aligned}\text{PROJS_BY_ROSSI} &\leftarrow \pi_{\text{Pno}}(\text{WORKS_ON} \bowtie_{\text{ESSN} = \text{SSN AND FNAME} = \text{ROSSI}} \text{EMPLOYEE}) \\ \text{REQUIREMENTS} &\leftarrow \text{CANDIDATES} \times \text{PROJS_BY_ROSSI}\end{aligned}$$

3. Using the actual information we have about every candidate, find the witnesses, i.e., single instances from the requirements that a candidate fails to meet (i.e., projects worked by by "ROSSI" but not by a candidate employee)

$$\begin{aligned}\text{ACTUAL_INFORMATION} &\leftarrow \pi_{\text{ESSN}, \text{Pno}}(\text{WORKS_ON}) \\ \text{WITNESSES} &\leftarrow \text{REQUIREMENTS} - \text{ACTUAL_INFORMATION}\end{aligned}$$

Query with universal conditions II

The case of "all" / subset, $\text{projects_by_rossi} \subseteq \text{projects_by_candidate}$ - operationally

4. Identify the candidates that have at least one witness showing that they are not good candidates

$$\text{NO_GOOD} \leftarrow \pi_{\text{ESSN}}(\text{WITNESSES})$$

5. Find the good candidates by removing no good candidates from all the candidates

$$\text{GOOD} \leftarrow \text{CANDIDATES} - \text{NO_GOOD}$$

Query with universal conditions

The case of "only" / superset, $\text{projects_by_rossi} \supseteq \text{projects_by_candidate}$ - operationally

1. Identify the set of candidates as before
2. Here, the set of requirements represents what at most (before it was at least) each candidate must satisfy in order to be considered a good candidate. The requirement set is the same as before, just different meaning

$$\text{REQUIREMENTS} \leftarrow \text{CANDIDATES} \times \text{PROJS_BY_ROSSI}$$

3. A witness here should represent an instance of the actual information that is not part of what is at most allowed by requirements (e.g., a project for which my candidate is working on but not ROSSI). We need to invert operands on the set difference

$$\text{ACTUAL_INFORMATION} \leftarrow \pi_{\text{ESSN}, \text{Pno}}(\text{WORKS_ON})$$

$$\text{WITNESSES} \leftarrow \text{ACTUAL_INFORMATION} - \text{REQUIREMENTS}$$

4. Identify no good and good candidates as before

$$\text{NO_GOOD} \leftarrow \pi_{\text{ESSN}}(\text{WITNESSES})$$

$$\text{GOOD} \leftarrow \text{CANDIDATES} - \text{NO_GOOD}$$

Query with universal conditions

All the other cases $\subset, \supset, =$ - operationally

- ▶ Concerning equality, e.g., $\text{projects_by_rossi} = \text{projects_by_candidate}$
 1. Find the candidates that satisfy $\text{projects_by_rossi} \subseteq \text{projects_by_candidate}$ condition
 2. Find the candidates that satisfy $\text{projects_by_rossi} \supseteq \text{projects_by_candidate}$ condition
 3. Candidates that satisfy equality ($=$) are those in the intersection $\cap$ of the two sets
- ▶ Concerning proper subset, e.g., $\text{projects_by_rossi} \subset \text{projects_by_candidate}$ (e.g., employee has all ROSSI projects plus at least one extra)
 1. Find the candidates that satisfy $\text{projects_by_rossi} \subseteq \text{projects_by_candidate}$ condition
 2. Find the candidates that satisfy $\text{projects_by_rossi} = \text{projects_by_candidate}$ condition
 3. Candidates that satisfy proper subset ($\subset$) are those that are in the first set but not in the second (via set difference)
- ▶ Concerning proper superset, e.g., $\text{projects_by_rossi} \supset \text{projects_by_candidate}$ (employee is missing at least one ROSSI project)
 1. Find the candidates that satisfy $\text{projects_by_rossi} \supseteq \text{projects_by_candidate}$ condition
 2. Find the candidates that satisfy $\text{projects_by_rossi} = \text{projects_by_candidate}$ condition
 3. Candidates that satisfy proper superset ($\supset$) are those that are in the first set but not in the second (via set difference)

Binary Relational Operations: Division

A compact way to express universal conditions

- ▶ Given $R(X, Y)$ and $S(Y)$, the division $R \div S$ returns the values of X that are related to **all** values in S
- ▶ In our example:

$\text{WORKS_ON}(\text{ESSN}, \text{Pno}) \div \text{PROJS_BY_ROSSI}(\text{Pno})$

returns the employees that work on **all** projects worked on by Rossi

- ▶ Therefore, division directly captures the condition:

$\text{projects_by_rossi} \subseteq \text{projects_by_candidate}$

- ▶ In this course, we prefer the witness-based construction, since it also generalizes to the other cases (and to SQL)

Complete Set of Relational Operations

- ▶ The set of operations including SELECT σ , PROJECT π , UNION $\cup$, DIFFERENCE $-$, RENAME ρ , and CARTESIAN PRODUCT $\times$ is called a *complete set* because any other relational algebra expression can be expressed by a combination of these five operations.
- ▶ For example:
 - ▶ $R \cap S = (R \cup S) - ((R - S) \cup (S - R))$
 - ▶ $R \bowtie_{\langle \text{join condition} \rangle} S = \sigma_{\langle \text{join condition} \rangle}(R \times S)$
 - ▶ $R \bowtie S = \pi_{\langle \text{union of R and S attributes} \rangle}(\sigma_{\langle \text{equality condition over shared attributes} \rangle}(R \times S))$

Complete Set of Relational Operations

Other Binary Relational Operations: SEMIJOIN and ANTISEMIJOIN

- ▶ The **left semijoin** of R with S , denoted by $R \ltimes S$, returns the tuples of R that **match at least one tuple** of S on the join attributes
- ▶ Intuitively:
 - ▶ first combine related tuples as in a NATURAL JOIN
 - ▶ then keep only the attributes of the left relation
- ▶ Hence, if $R(\mathcal{X})$ and $S(\mathcal{Y})$, then:

$$R \ltimes S = \pi_{\mathcal{X}}(R \bowtie S)$$

- ▶ **Left anti-semijoin** returns the tuples of R that **do not match any tuple** of S
- ▶ It can be expressed by using semijoin and set difference:

$$R \rhd S = R - (R \ltimes S)$$

- ▶ Therefore:
 - ▶ $R \ltimes S$ keeps the tuples of R that **have a match**
 - ▶ $R \rhd S$ keeps the tuples of R that **have no match**
- ▶ Variants: Right semijoin $R \ltimes S$ and right anti-semijoin $R \rhd S$

Aggregate Functions and Grouping

- ▶ A type of request that cannot be expressed in the basic relational algebra is to specify mathematical **aggregate functions** on collections of values from the database
- ▶ Examples of such functions include retrieving the average or total salary of all employees or the total number of employee tuples
 - ▶ These functions are used in simple statistical queries that summarize information from the database tuples
- ▶ Common functions applied to collections of numeric values include
 - ▶ **SUM**, **AVERAGE**, **MAXIMUM**, and **MINIMUM**
- ▶ The **COUNT** function is used for counting tuples or values

Aggregate Function Operation

Use of the Aggregate Functional operation $\mathcal{F}_{\langle \text{function list} \rangle}$ (calligraphic capital F)

- ▶ $\mathcal{F}_{\text{MAX Salary}}(\text{EMPLOYEE})$ retrieves the maximum salary value from the EMPLOYEE relation
- ▶ $\mathcal{F}_{\text{MIN Salary}}(\text{EMPLOYEE})$ retrieves the minimum salary value from the EMPLOYEE relation
- ▶ $\mathcal{F}_{\text{SUM Salary}}(\text{EMPLOYEE})$ retrieves the sum of the Salary from the EMPLOYEE relation
- ▶ $\mathcal{F}_{\text{COUNT SSN, AVERAGE Salary}}(\text{EMPLOYEE})$ computes the count (number) of employees and their average salary
 - ▶ Note: count just counts the number of rows, without removing duplicates

Using Grouping with Aggregation

- ▶ The previous examples all summarized one or more attributes for a set of tuples
 - ▶ Maximum Salary or Count (number of) SSN
- ▶ **Grouping** can be combined **with Aggregate Functions**
- ▶ Example: For each department, retrieve the DNO, COUNT SSN, and AVERAGE SALARY
- ▶ A variation of aggregate operation $\langle \text{grouping attributes} \rangle \mathcal{F} \langle \text{function list} \rangle$:
 - ▶ Grouping attribute placed to left of $\mathcal{F}$
 - ▶ Aggregate functions to right of $\mathcal{F}$
 - ▶ Degree of the resulting relation: $|\langle \text{grouping attributes} \rangle| + |\langle \text{function list} \rangle|$
 - ▶ $\text{DNO} \mathcal{F} \text{COUNT SSN, AVERAGE Salary}(\text{EMPLOYEE})$
- ▶ Above operation groups employees by DNO (department number) and computes the count of employees and average salary per department

Using Grouping with Aggregation

Graphical example

$$R(\text{Dno}, \text{No_of_employees}, \text{Average_sal}) \leftarrow \text{Dno} \mathcal{F}_{\text{COUNT Ssn, AVERAGE Salary}}(\text{EMPLOYEE})$$

Dno	No_of_employees	Average_sal
5	4	33250
4	3	31000
1	1	55000

$$\text{Dno} \mathcal{F}_{\text{COUNT Ssn, AVERAGE Salary}}(\text{EMPLOYEE})$$

Dno	Count_ssn	Average_salary
5	4	33250
4	3	31000
1	1	55000

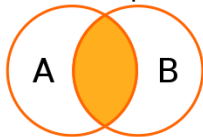
$$\mathcal{F}_{\text{COUNT Ssn, AVERAGE Salary}}(\text{EMPLOYEE})$$

Count_ssn	Average_salary
8	35125

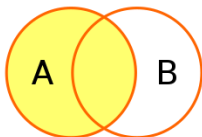
OUTER JOIN Operation

- ▶ In NATURAL JOIN and EQUIJOIN, **tuples without a matching** (or related) tuple **are eliminated** from the join result
 - ▶ Tuples with *null* in the join attributes are also eliminated
 - ▶ This amounts to loss of information.
- ▶ A set of operations, called **OUTER joins**, can be used when we want to **keep all the tuples in R , or all those in S , or all those in both relations** in the result of the join, regardless of whether or not they have matching tuples in the other relation.

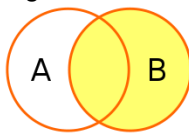
Natural / Equi Join



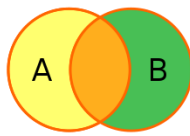
Left Outer Join



Right Outer Join



Full Outer Join



OUTER JOIN Operation (cont.)

- ▶ The **left outer join** operation $R \bowtie S$ keeps *every tuple* in the first or left relation R ; if no matching tuple is found in S , then the attributes of S in the join result are filled or “padded” with `null` values.
- ▶ A similar operation, **right outer join**, keeps every tuple in the second or right relation S in the result of $R \bowtie S$, padding with `null` the attributes of R if needed.
- ▶ A third operation, **full outer join**, denoted by $R \bowtie S$, keeps all tuples in both the left and the right relations when no matching tuples are found, padding them with `null` values as needed.

OUTER JOIN Operation (cont.)

Example

- NL List all employee names as well as the name of the department they manage if they happen to manage a department

OUTER JOIN Operation (cont.)

Example

NL List all employee names as well as the name of the department they manage if they happen to manage a department

RA

$$\text{TEMP} \leftarrow (\text{EMPLOYEE} \bowtie_{\text{SSN}=\text{MGR_SSN}} \text{DEPARTMENT})$$
$$\text{RESULT} \leftarrow \pi_{\langle \text{FNAME}, \text{MINIT}, \text{LNAME}, \text{DNAME} \rangle}(\text{TEMP})$$

RESULT

Fname	Minit	Lname	Dname
John	B	Smith	NULL
Franklin	T	Wong	Research
Alicia	J	Zelaya	NULL
Jennifer	S	Wallace	Administration
Ramesh	K	Narayan	NULL
Joyce	A	English	NULL
Ahmad	V	Jabbar	NULL
James	E	Borg	Headquarters

Exercises

1. Return the name and the surname of all employees
2. Return the SSN of female employees with a salary greater than 30k \$
3. Find the employees who do not have any dependents
4. Return the SSN of employees who manage a department
5. Return the SSN, name and surname of employees who manage a department together with the name of the department they manage
6. Find those employees whose salary is greater than that of their supervisors
7. Find those employees that belong to the same department
8. Return the employees that have at least two dependents
9. Return the employees that have at most two dependents
 - a. Return the employees that have at least one and at most two dependants
10. Find the employees that work on **all** projects employee "ROSSI" works on.
 - a. Find the employees that work **only** on projects employee "ROSSI" works on
 - b. Find the employees that work **all and only** on projects employee "ROSSI" works on

Exercise

Let the following relational scheme concerning films and actors be given:

- ▶ FILM(FilmCode, Title, Director, Year);
- ▶ ACTOR(ActorCode, Surname, FirstName, Sex, BirthDate, Nationality);
- ▶ INTERPRETATION(Film, Actor, Character)

Each film is identified by a code and characterised by a title, a director and the release year. Let us assume that each film is directed by a single director, which is uniquely identified by his surname. We do not exclude the possibility of different films having the same title. Each actor is uniquely identified by a code and characterised by a first name, a surname, a gender, a date of birth and a nationality. Several actors may act in the same film and one actor may act in several films. Let us assume that an actor can only play one character in a film. First define the primary keys, any other candidate keys and, if there are any, the foreign keys of the given relations. Next, formulate appropriate queries in relational algebra to determine (without using the division operator and using aggregate functions only if necessary)

1. the actors who have acted in all the films directed by the director Antonioni;
2. the actors who have acted in at least two films directed by director Allen (then, the the at most case);
3. the pairs of actors such that there is a film in which the first acted and the second did not, and vice versa.

References

Material for these slides is largely based on:

- ▶ Elmasri and Navathe, *Fundamentals of Database Systems*, 7th Edition, Chapter 8: "The Relational Algebra and Relational Calculus" (book chapter and accompanying slides)